A MAJOR PROJECT REPORT

ON

ENHANCEMENT OF VIRTUAL ASSISTANTS THROUGH MULTIMODAL AI FOR EMOTION RECOGNITION

Submitted to

SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

In partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted

By

M. VISHAL REDDY - (22D41A7265)

M. MOHAN - (22D41A7276)

SHAIK SOHEL - (22D41A72A7)

V. NAGENDAR REDDY - (22D41A72C1)

Under the esteemed guidance of

DR.T.CHARAN SINGH

(Assistant Professor)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC, accredited by NBA, Affiliated to JNTUH)

Sheriguda,Ibrahimpatnam.

(2025-2026)

SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC, Accredited by NBA, Affiliated to JNTUH)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CERTIFICATE

Certified that the Major Project entitled **"ENHANCEMENT OF VIRTUAL ASSISTANTS THROUGH MULTIMODAL AI FOR EMOTION RECOGNITION"** is a Bonafide work carried out **by M.VISHAL REDDY (22D41A7265), M.MOHAN(22D41A7276),SHAIK.SOHEL(22D41A72A7), V. NAGENDAR REDDY (22D41A72C1)** in partial fulfillment for the award of **Bachelor of Technology** in **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE** of **SICET**, Hyderabad for the academic year 2025- 2026. The Project has been approved as it satisfies academic requirements in respect of the work prescribed for **IV YEAR, II-SEMESTER** of **B. TECH** course.

INTERNAL GUIDE

(Dr. T. Charan Singh)

(Head of the department)

HEAD OF THE DEPARTMENT

(Dr. T. Charan Singh)

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

With great pleasure we want to take this opportunity to express our heartfelt gratitude to all the people who helped in making this project work a success. We thank the almighty forgiving us the courage & perseverance in completing the project.

We are thankful to principal **Prof. Dr. G. Suresh**, for giving us the permission to carry out this project and for providing necessary infrastructure and labs.

We are highly indebted to, **Dr. T. Charan Singh**, Head of the Department of Artificial Intelligence and Data Science, Project Guide, for providing valuable guidance at every stage of this project.

We are grateful to our internal project guide, **Dr. T. Charan Singh** for his constant motivation and guidance given by his during the execution of this project work.

We would like to thank the Teaching & Non-Teaching staff of Department of Artificial Intelligence and Data Science for sharing their knowledge with us.

Last but not the least we express our sincere thanks to everyone who helped directly or indirectly for the completion of this project.

M. VISHAL REDDY - (22D41A7265)

M. MOHAN - (22D41A7276)

SHAIK SOHEL - (22D41A72A7)

V. NAGENDAR REDDY - (22D41A72C1)

INDEX

S.NO	TOPICS	PAGE NO.
1	**INTRODUCTION**	1-2
2	**LITERATURE SURVEY**	3-4
3	**SYSTEM ANALYSIS** 3.1 Existing System 3.2 Proposed System	5-6
4	**SYSTEM REQUIREMENTS** 4.1 Functional Requirements 4.2 Non-Functional Requirements	7-8
5	**SYSTEM STUDY** 5.1 Feasibility Study 5.2 Feasibility Analysis	9-11
6	**SYSTEM DESIGN** 6.1 System Architecture 6.2 UML Diagrams 6.2.1 Use Case Diagram 6.2.2 Class Diagram 6.2.3 Sequence Diagram 6.2.4 Collaboration Diagram 6.2.5 Activity Diagram 6.2.6 Component Diagram 6.2.7 Deployment Diagram 6.2.8 ER Diagram 6.2.9 Data Dictionary	12-24
7	**INPUT AND OUTPUT DESIGN** 7.1 Input Design 7.2 Output Design	25-26
8	**IMPLEMENTATION** 8.1 MODULES	27-29

	8.2 Module Description	
9	**SOFTWARE ENVIRONMENT** 9.1 Java 9.2 Source Code	30-59
10	**RESULTS/DISCUSSIONS** 10.1 System Testing 10.2 Test Cases 10.3 Screenshots	60-80
11	**CONCLUSION** 11.1 Conclusion 11.2 Future Scope	81-82
12	**REFERENCES/BIBLIOGRAPHY**	83

LIST OF FIGURES

S.NO	TABLES/FIGURES	PAGE NO
1	System Architecture	12
2	UML Diagrams	
	2.1 Use Case Diagram	14-15
	2.2 Class Diagram	16
	2.3 Sequence Diagram	17
	2.4 Collaboration Diagram	18
	2.5 Activity Diagram	19
	2.6 Component Diagram	20
	2.7 Deployment Diagram	21
	2.8 ER Diagram	22
	2.9 Data Dictionary	23-24
3	Screenshots	66-80

ENHANCEMENT OF VIRTUAL ASSISTANTS THROUGH MULTIMODAL AI FOR EMOTION RECOGNITION

ABSTRACT

Emotion recognition is becoming increasingly critical for enhancing human-computer interactions, as emotions play a vital role in shaping human interactions and overall well-being. Machines that can detect and respond to emotional cues similar to humans are essential in multiple industries. Emotionally responsive agents find applications in education, healthcare, gaming, marketing, customer service, humanrobot interaction, and entertainment. This study explores the potential of enhancing virtual assistants through multimodal Artificial Intelligence (AI), utilizing various emotion recognition techniques to create more empathetic and effective systems. The proposed methodology makes use of facial expressions and textual cues to enhance the emotional awareness of the system and achieve user satisfaction through empathetic conversation.

The Facial Emotion Recognition (FER) model achieved 71% real-time accuracy, whereas the Textual Emotion Recognition (TER) model achieved 59% validation accuracy, demonstrating effective Multimodal Emotion Recognition (MER). Unlike prior multimodal emotion-aware systems, our lightweight architecture ensures real-time inference and uniquely integrates facial and textual emotion recognition with DialoGPT-based response generation — demonstrating compatibility with large language models for empathetic dialogue.

KEYWORDS:

1. INTRODUCTION

The rapid advancement of Artificial Intelligence (AI) has significantly transformed the way humans interact with machines. From rule-based chatbots to highly intelligent conversational agents powered by deep learning, virtual assistants have evolved to become an integral part of daily life. However, despite their growing intelligence, most existing virtual assistants lack one critical human capability — the ability to understand and respond appropriately to emotions. Human communication is deeply influenced by emotional states, and effective interaction depends not only on what is said, but also on how it is expressed. Therefore, enabling machines to recognize and respond to emotions is essential for creating more natural, empathetic, and human-like interactions.

Emotion recognition refers to the process of identifying human emotions through observable cues such as facial expressions, speech tone, gestures, and textual content. Traditional systems have primarily focused on single-modal approaches, such as Facial Emotion Recognition (FER) or Textual Emotion Recognition (TER). While these approaches have shown promising results individually, they often fail to capture the full complexity of human emotions. Emotions are multi-dimensional and context-dependent; a smile may not always indicate happiness, and textual sentiment may not always reflect true emotional intent. Relying on a single input source can therefore reduce accuracy and limit the system's ability to interpret subtle or mixed emotional states.

To address these limitations, Multimodal Emotion Recognition (MER) has emerged as a powerful approach that combines multiple data modalities — such as visual and textual inputs — to improve emotion detection performance. By integrating facial expressions with textual cues, systems can achieve a more comprehensive understanding of user emotions. This multimodal strategy enhances robustness, reduces ambiguity, and increases detection accuracy compared to unimodal systems. The fusion of features from different modalities allows the system to compensate when one modality provides incomplete or unclear information.

In this project, titled **"Enhancement of Virtual Assistants Through Multimodal AI for Emotion Recognition,"** we propose a lightweight and real-time emotion-aware virtual assistant that integrates both FER and TER models. The facial emotion recognition component analyzes facial landmarks and expression patterns using deep learning techniques, while the textual emotion recognition component processes user

input using natural language processing (NLP) methods. The extracted features from both modalities are fused to perform accurate emotion classification. Furthermore, the system incorporates a DialoGPT-based response generator to produce empathetic and context-aware replies, enabling more meaningful human-computer interactions.

The proposed system emphasizes real-time performance and computational efficiency, making it suitable for deployment in practical environments such as education, healthcare, customer service, gaming, and human-robot interaction. By generating emotionally adaptive responses, the system aims to improve user satisfaction, engagement, and trust in AI-based assistants.

In conclusion, this project contributes to the development of emotionally intelligent virtual assistants by leveraging multimodal AI techniques. By bridging the emotional gap between humans and machines, the proposed system moves toward creating more natural, empathetic, and intelligent interactive systems that better understand and respond to human needs.

2. LITERATURE SURVEY

TITLE: Facial Emotion Recognition Using Deep Convolutional Neural Networks

AUTHORS: Li, S., Deng, W. (2020)

ABSTRACT:

This study investigates the use of Deep Convolutional Neural Networks (CNNs) for Facial Emotion Recognition (FER). The research demonstrates that CNN-based models can automatically extract facial features such as landmarks, muscle movements, and micro-expressions to classify emotions like happiness, sadness, anger, fear, and surprise. The model achieves high accuracy on benchmark datasets such as FER-2013 and CK+. The study highlights that deep learning eliminates the need for manual feature engineering. However, limitations include reduced performance in real-time environments, sensitivity to lighting conditions, and difficulty in detecting subtle or mixed emotions.

TITLE: Text-Based Emotion Recognition Using Transformer Models

AUTHORS: Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019)

ABSTRACT:

This research introduces transformer-based models for natural language understanding tasks, including sentiment and emotion analysis. Using contextual embeddings, transformer models such as BERT significantly improve textual emotion recognition (TER) by capturing semantic and contextual meaning. The study demonstrates improved performance over traditional machine learning approaches. However, limitations include high computational requirements and challenges in detecting sarcasm, ambiguity, or mixed emotional states in conversational text.

TITLE :**Multimodal Emotion Recognition: A Survey**

AUTHORS: Poria, S., Cambria, E., Bajpai, R., & Hussain, A. (2017)

ABSTRACT:

This survey explores multimodal emotion recognition (MER) systems that combine facial, textual, and speech modalities. The study concludes that multimodal systems outperform unimodal systems due to complementary information across modalities. Fusion strategies such as feature-level and decision-level fusion are discussed. While multimodal systems

improve robustness and accuracy, challenges include synchronization of modalities, increased computational complexity, and real-time deployment constraints.

TITLE: DialoGPT: Large-Scale Generative Pre-training for Conversational Response Generation

AUTHORS: Zhang, Y., Sun, S., Galley, M., et al. (2020)

ABSTRACT:
This paper introduces DialoGPT, a large-scale pretrained conversational model based on transformer architecture. The model generates human-like and contextually relevant responses in dialogue systems. Experimental results show improved conversational coherence compared to traditional chatbot systems. However, limitations include lack of emotional awareness and potential generation of neutral or generic responses without additional emotion-conditioning mechanisms.

TITLE: Emotion-Aware Chatbots for Improved Human-Computer Interaction

AUTHORS: Rashkin, H., Smith, E. M., Li, M., & Boureau, Y. L. (2019)

ABSTRACT:
This research focuses on building empathetic dialogue systems capable of recognizing and responding to emotions in conversation. The study demonstrates that incorporating emotion labels into conversational models improves response empathy and user satisfaction. While emotion-aware chatbots show improved interaction quality, challenges include reliable emotion detection and real-time inference efficiency.

TITLE: Real-Time Facial Emotion Recognition in Human-Computer Interaction

AUTHORS: Happy, S. L., & Routray, A. (2015)

ABSTRACT:
This paper proposes a real-time FER system using facial landmark detection and machine learning classifiers. The model demonstrates practical deployment capability in interactive applications. Results indicate acceptable accuracy in controlled environments. However, performance drops in real-world scenarios due to occlusion, pose variation, and illumination changes. The study highlights the need for more robust and lightweight models for real-time systems.

TITLE: Multimodal Fusion Techniques for Emotion Recognition

AUTHORS: Atrey, P. K., Hossain, M. A., El Saddik, A., & Kankanhalli, M. S. (2010)

ABSTRACT:
This research examines different multimodal fusion techniques including early fusion, late fusion, and hybrid fusion approaches. The study concludes that feature-level fusion often improves classification performance when modalities are complementary. However, improper fusion strategies may introduce noise and reduce accuracy. The paper emphasizes the importance of lightweight and efficient fusion mechanisms for real-time applications.

TITLE: Emotion Recognition in Human-Computer Interaction: Challenges and Opportunities

AUTHORS: Calvo, R. A., & D'Mello, S. (2010)

ABSTRACT:
This study explores the importance of emotion recognition in enhancing human-computer interaction (HCI). It highlights how emotionally intelligent systems improve user engagement, trust, and satisfaction. The paper identifies challenges such as ambiguity in emotional expression, privacy concerns, and ethical considerations in emotion data collection. The authors suggest that multimodal AI systems provide a promising solution for improving emotional intelligence in virtual assistants.

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Currently existing systems for emotion recognition are limited to using either facial expression recognition or textual sentiment analysis (or some other single-modal approach) to detect emotion to some extent. The existing systems often struggle to detect context and utilize for subtle or complex cues of emotions. Many systems are also computational heavy, resulting in longer and slower real-time inference times, and don't generate empathetic or context-effect responses and conversations about end-users interactions.

DISADVANTAGES

1. Only providing emotion detection, and not multimodal detection, resulting in low accuracy.
2. Poor performance when detecting subtle or mixed emotions.
3. Longer inference times for real-time applications.
4. Poor or low empathetic response generation for the user.
5. Low flexibility for performance flexibility across environments, and input types (sounds, text and visual).

3.2 PROPOSED SYSTEM

This proposal proposes a Multimodal Emotion Recognition (MER) methodology, which captures and combines facial expressions and text signals to provide accurate and holistic emotion detection. In addition, the combination of Facial Emotion Recognition (FER) and Textual Emotion Recognition (TER) models with a DialoGPT-based response generator allows the system to perceive and sense user emotions in real-time while also generating an empathetic response. Its lightweight infrastructure ensures rapid inferences while still preserving sufficient accuracy for detecting user emotions.

ADVANTAGES

1. Benefits from multimodal inputs (facial + textual) for improved accuracy.
2. Ability to recognize emotions in real-time and a lightweight system.
3. Has the capacity to generate a empathetic and contextual response.
4. Increased user satisfaction in human-computer interaction.
5. Flexible to usage in multiple domains like education, health care, gaming, customer service.

SYSTEM REQUIREMENTS

3.3 FUNCTIONAL REQUIREMENTS:

Functional requirements describe the core features and services that the Multimodal Emotion Recognition (MER) Virtual Assistant must provide.

USER MODULES:

1. User Registration and Login

•**Secure Authentication:** The system shall allow users to register and log in using secure credentials (username/email and password).

•**Role-Based Access Control:** The system shall support role-based access (User, Researcher, Admin) to control permissions for emotion analysis, reports, and system management.

• **Data Protection:** The system shall encrypt passwords and sensitive user data to ensure privacy and secure storage of emotional interaction records.

2. Accessing the Emotion-Aware Dashboard

• **Personalized Dashboard:** The system shall provide a dashboard displaying detected emotions, interaction history, and system performance metrics.

• **Real-Time Emotion Display:** The system shall show detected facial and textual emotions instantly after analysis.

• **Interaction History:** Users shall be able to view past conversations and emotion analysis summaries.

3. Facial Data Acquisition and Processing

• **Webcam Integration:** The system shall capture real-time facial images through a webcam.

• **Image Upload Option:** Users shall be able to upload facial images for emotion analysis.

• **Preprocessing:** The system shall normalize, resize, and process images before feeding them into the

FER model.

• **Face Detection:** The system shall detect and isolate facial regions automatically before classification.

4. Textual Data Processing

• **Text Input:** The system shall accept user text messages for emotion analysis.

• **Text Preprocessing:** The system shall clean, tokenize, and convert text into embeddings suitable for the TER model.

• **Context Analysis:** The system shall analyze semantic and emotional tone using NLP techniques.

5. Multimodal Emotion Recognition (MER)

• **Feature Extraction:** The system shall extract emotional features from both facial and textual inputs.

• **Feature Fusion:** The system shall combine facial and textual features using feature-level fusion to improve classification accuracy.

• **Emotion Classification:** The system shall classify emotions into predefined categories such as happy, sad, angry, surprised, neutral, etc.

• **Real-Time Prediction:** The system shall provide emotion predictions within seconds for interactive applications.

6. Empathetic Response Generation

• **Emotion-Aware Response:** The system shall generate context-aware and emotionally appropriate responses using a DialoGPT-based conversational model.

• **Supportive Interaction:** If negative emotions (e.g., sadness or stress) are detected, the system shall generate supportive and empathetic replies.

• **Context Continuity:** The system shall maintain conversational context for meaningful dialogue.

7. Performance Evaluation and Feedback

• **Model Evaluation Metrics:** The system shall compute accuracy, precision, recall, and F1-score for FER and TER models.

• **User Feedback Collection:** Users shall be able to provide feedback on system responses to improve future performance.

• **Performance Monitoring:** The system shall monitor latency and response time for real-time efficiency.

ADMIN MODULES:

1. User Management

• Administrators shall create, update, or delete user accounts.

• The system shall log login history and activity for monitoring purposes.

2. Model Management

• Administrators shall update FER, TER, and response generation models.

• The system shall allow retraining or fine-tuning of AI models when required.

• Performance metrics shall be accessible for evaluation.

3. Data and Security Management

• Administrators shall monitor stored emotional interaction data.

• The system shall enforce data privacy policies and secure storage mechanisms.

• Audit logs shall record system access and updates.

4. System Monitoring

• The system shall monitor server load and inference time.

• Alerts shall be generated for unusual system activity or failures.

5. Reporting and Analytics

• Administrators shall generate reports on system usage and emotion detection trends.

• Analytics shall support performance improvements and optimization decisions.

NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes of the system.

1. Performance

• The system shall perform real-time emotion detection within a few seconds.

• The response generation shall occur with minimal latency.

2. Scalability

• The system shall support multiple concurrent users.

• It shall allow deployment in various domains such as healthcare, education, and customer service.

3. Security

• All sensitive data shall be encrypted during storage and transmission.

• Role-based authentication shall prevent unauthorized access.

4. Reliability and Availability

• The system shall maintain high availability during operational hours.

• Backup and recovery mechanisms shall prevent data loss.

5. Usability

• The interface shall be simple, interactive, and user-friendly.

• Minimal technical expertise shall be required to operate the system.

6. Maintainability

• The architecture shall be modular.

• FER, TER, and response generation components shall be updated independently without affecting other modules.

3.3.1 HARDWARE REQUIREMENTS

- System : Pentium IV 2.4 GHz Processor
- Hard Disk : 40 GB
- Ram : 512 MB

3.3.2 SOFTWARE REQUIREMENTS

- Operating System : Windows / Linux
- Programming Language : Python
- AIML Libraries : TensorFlow, PyTorch,OpenCV, Numpy, Pandas, HuggingFace Transformers
- Front End Technologies : HTML5, CSS3, Java Script
- Database : MYSQL / PostgreSQL

4. SYSTEM DESIGN

4.1 FEASIBILITY STUDY

The feasibility of the Enhancement of Virtual Assistants Through Multimodal AI for Emotion Recognition is analyzed in this phase and a business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the organization. For feasibility analysis, some understanding of the major requirements for the system is essential.

4.2 FEASIBILITY ANALYSIS

Three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system is well within the budget and this was achieved because most of the technologies used are freely available. Open-source frameworks such as TensorFlow, PyTorch and HuggingFace Transformers eliminate licensing costs significantly. Only the customized deployment infrastructure and edge server hardware components had to be provisioned, keeping the overall investment minimal and justified.

TECHNICAL FEASIBILITY

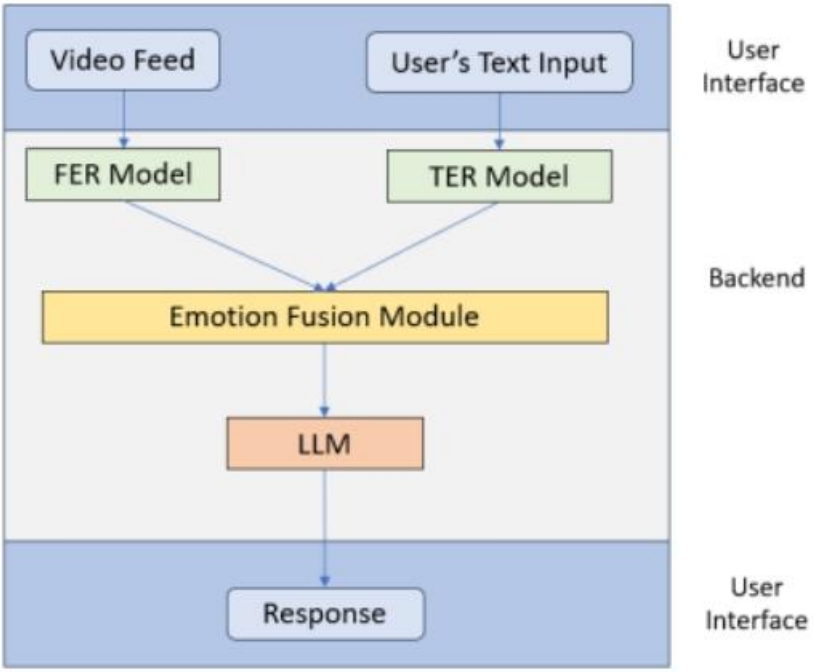
This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system. The proposed framework relies on well-established and readily available technologies, ensuring that the technical prerequisites are easily met without imposing excessive resource demands.

SOCIAL FEASIBILITY

The aspect of this study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make them familiar with it. Their level of confidence must be raised so that they are also able to make some constructive criticism, which is welcomed, as they are the final users of the system.

5. SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE:



5.2 UML DIAGRAMS

UML stands for Unified Modelling Language. UML is a standardized general-purpose modelling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta- model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

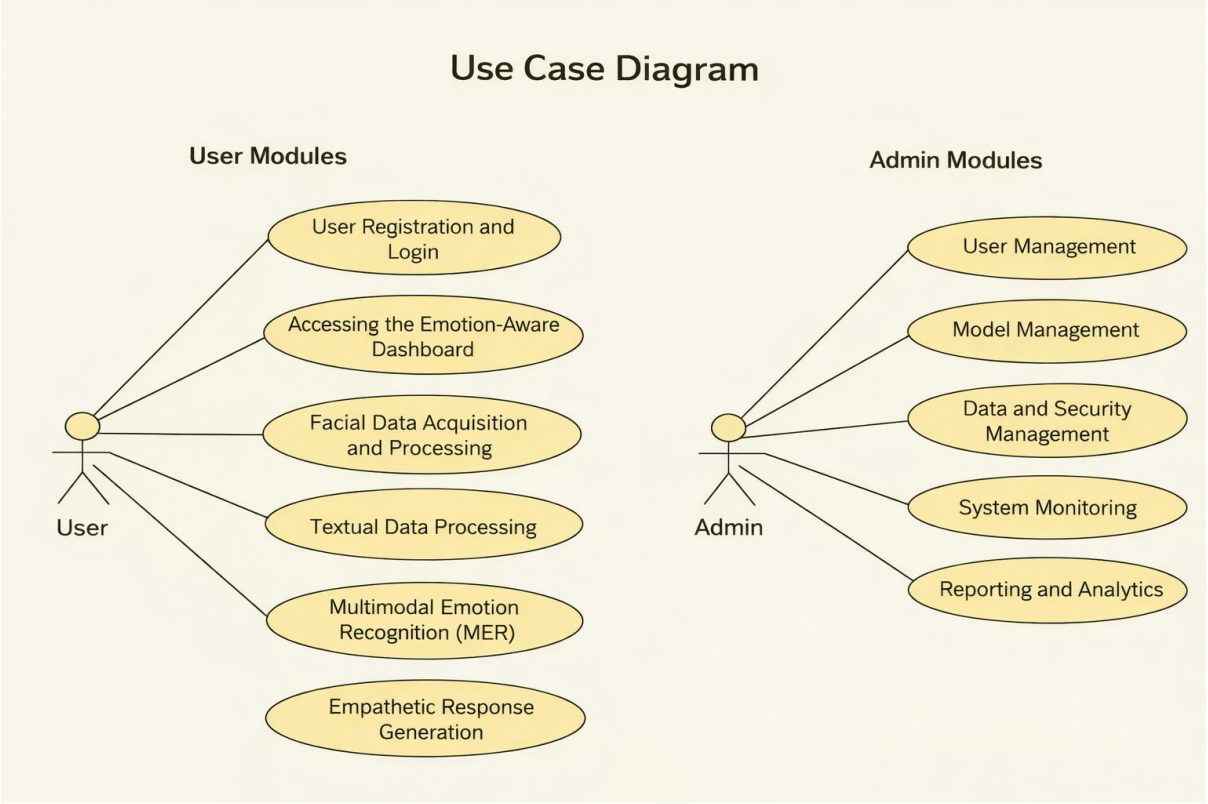
GOALS:

The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modelling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts. Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modelling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

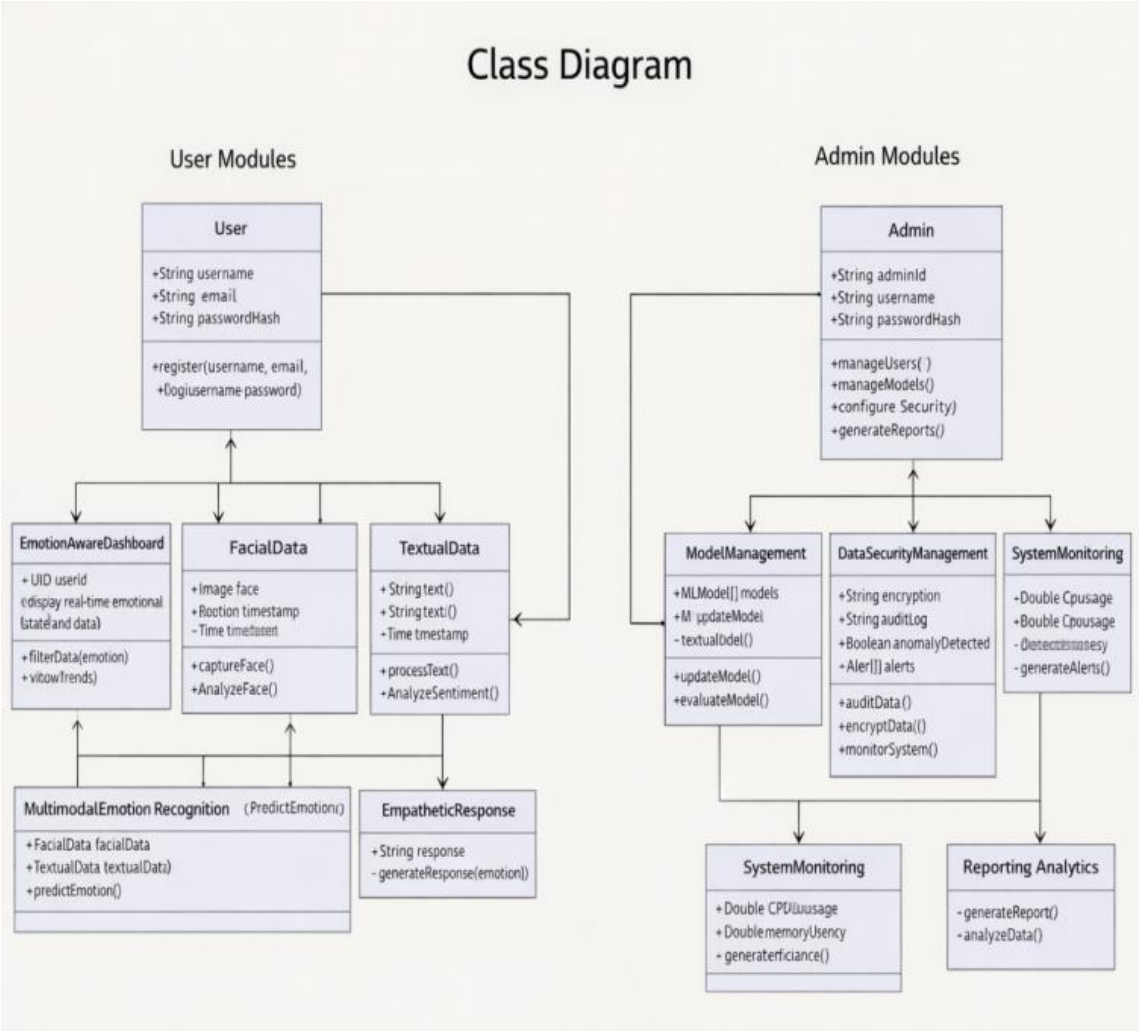
5.2.1 USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



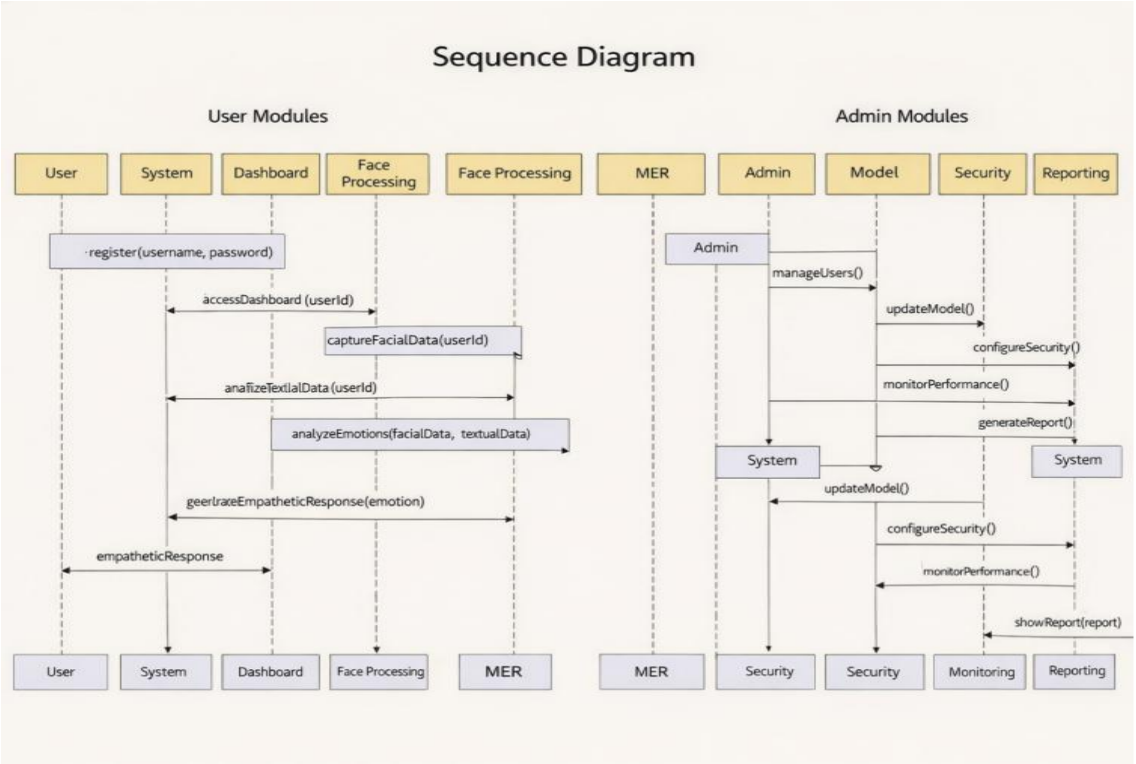
5.2.2 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.



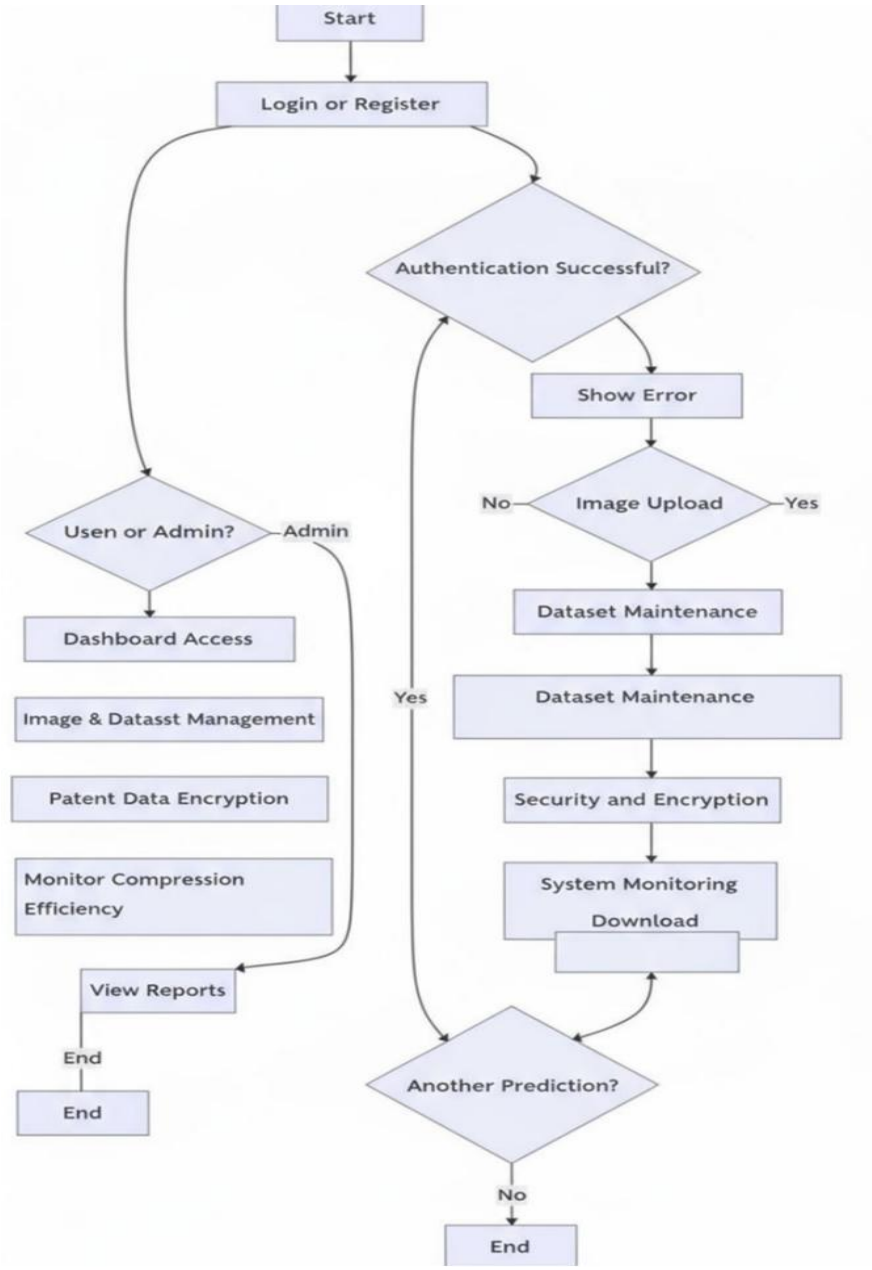
5.2.3 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.



5.2.4 ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



6. INPUT AND OUTPUT DESIGN

6.1 INPUT DESIGN

Considering the requirements of the Multimodal Emotion Recognition (MER) system, input design is structured to efficiently capture and process both facial and textual data. The system accepts real-time facial inputs through a camera for emotion detection and textual inputs through user interaction interfaces such as chat or speech-to-text modules. The design ensures minimal latency, simplicity in user interaction, and accurate data acquisition, enabling seamless integration of multimodal inputs for effective emotion recognition and response generation.

INFO style

The input configuration represents the interaction between the Multimodal Emotion Recognition (MER) system and the user. It involves designing efficient methods for capturing and processing multimodal data, ensuring that facial and textual inputs are transformed into a usable format for emotion analysis. The required data is obtained either through real-time camera input for facial expressions or through user-generated text via chat interfaces, while maintaining simplicity, accuracy, and minimal latency. The design of input focuses on controlling data volume, reducing errors, avoiding delays, eliminating redundancy, and ensuring a seamless user experience. Additionally, it emphasizes privacy and security while handling sensitive user information such as facial data and personal text inputs. Input design considered the following aspects:

What data should be provided as input?

1. **Data Organization and Encoding:** The system structures input data by capturing facial frames and preprocessing them for the ViT-Tiny model, while textual inputs are tokenized and encoded for the MiniLM model to extract contextual emotional features.

2. **Input Validation and Error Handling:** Mechanisms are implemented to validate inputs, such as ensuring proper face detection, handling missing or unclear facial data, and managing incomplete or ambiguous text inputs, with fallback strategies for error conditions.

3. **Objective of Input Design:** The goal is to convert user-provided multimodal data into a structured format suitable for computational processing, minimizing errors and ensuring reliable emotion recognition within the system.

4. **User Interface and Data Entry:** The system provides simple and intuitive interfaces for capturing facial and textual inputs, enabling efficient handling of continuous data streams while maintaining accuracy and ease of use.

5. **Performance Considerations:** Input design ensures that data acquisition does not introduce unnecessary delays, supporting real-time processing in dynamic environments such as conversational AI systems, virtual assistants, and interactive applications.

11. 2 OUTPUT DESIGN YIELD STYLE

A quality output is one that meets user requirements and presents information clearly and effectively. In the Multimodal Emotion Recognition (MER) system, the results of processing are communicated to users and other system components through well-structured outputs. Output design determines how the recognized emotions and generated responses are presented for immediate use as well as for logging and analysis. It serves as the primary source of information for the user, and an effective output design improves system usability and supports better interaction and decision-making.

1. **Structured Output Generation:** Output should be designed in a clear and organized manner, ensuring that all components such as detected facial emotions, textual emotion analysis, and final fused emotion are accurately presented. The system must generate outputs that support real-time interaction, including emotion labels, confidence scores, and system-generated empathetic responses.
2. **Methods of Displaying Information:** The system provides outputs through user-friendly interfaces such as dashboards or chat windows, where emotions and responses are displayed in an intuitive format. Visual indicators, text responses, and optional graphical representations may be used to enhance understanding.
3. **Reports and Output Formats:** The system can generate logs and reports containing emotion history, interaction records, and system responses. These outputs can be stored for analysis, performance evaluation, or future reference, ensuring flexibility in different application scenarios.

7. IMPLEMENTATION

7.1 MODULES

- USER REGISTRATION AND LOGIN MODULE
- CONTENT MANAGEMENT MODULE
- AI INTEGRATION MODULE
- ASSESSMENT MODULE
- DASHBOARD MODULE

7.2 MODULE DESCRIPTION

1. **Acquisition and Preprocessing of Data:**

This module is focused on acquiring and preprocessing the input data for emotion recognition. Facial images would be captured via a web camera or via uploaded images, both of which could be used alongside textual input including user messages or transcripts. Facial images normalized, resized, and may be augmented for additional model robustness. Textual data would be cleaned to remove excess noise, stop words, punctuation, and include tokenization and embedding. All data would be preprocessed for standardized feature extraction across all modalities.

2. **Feature Extraction:**

The module extracts meaningful cues of emotion from both facial data and textual data. For facial data, the Facial Emotion Recognition (FER) model extracts facial landmarks, expressions, and patterns to detect emotions including happiness, anger, sadness, and surprise. Textual data includes a Textual Emotion Recognition (TER) model with natural language processing (NLP) methods for extracting sentiment, emotional tone, and context. The dual feature extraction module ensures both modalities contribute to a holistic understanding of emotion.

3. **Classification of Emotion:**

Features extracted from both modality will be fused together to execute multimodal emotion recognition. The computation will be a combination of facial features and textual features (based) on a fusion strategy that will improve the accuracy of emotion detection

than either modality used independently. The classification module produces predicted emotions in real-time to allow responses to be generated directly.

4. **Response Generation:**

Once an emotion is recognized from a user, an appropriate and empathic reply is generated using the DialoGPT-based response generator by either replying with contextual relevance, followed by supportive interaction or replying with engaging interaction based on the interaction context. This results in a more satisfactory user experience and human-like interaction.

5. **Evaluation and Feedback:**

This section evaluates overall behavior based on metrics such as accuracy, precision, recall, and F1-score across both FER and TER models. Feedback from users is required to continually improve response generation and emotion recognition. Monitoring is required in real time to ensure lower latency and responsive behavior across various applications.

8.SOFTWARE ENVIRONMENT

8.1 Python Technology

Python technology is both a programming language and a platform. The Python programming language is a high-level language that can be characterized by all of the following buzzwords:

- Simple
- Architecture neutral
- Object oriented
- Portable
- Distributed
- High performance
- Interpreted
- Multithreaded
- Robust
- Dynamic
- Secure

With most programming languages, you either compile or interpret a program so that you can run it on your computer. The Python programming language is unusual in that a program is both written in human-readable syntax and executed through an interpreter that translates it into machine-level instructions at runtime. Python source code is first compiled into an intermediate form called Python byte codes - the platform-independent instructions interpreted by the Python Virtual Machine (Python VM).

You can think of Python byte codes as the machine code instructions for the Python Virtual Machine. Every Python interpreter, whether it is a development environment, a command-line tool, or an embedded runtime within a server application, is an implementation of the Python VM. Python byte codes help make cross-platform development possible. You can write your program on any platform that has a Python interpreter installed. The byte codes can then be run on any implementation of the Python VM. That means that as long as a computer has a Python runtime, the same program written in the Python programming language can run on Windows, a Linux edge server, or a MacOS development machine.

The Python Platform

A platform is the hardware or software environment in which a program runs. We have already mentioned some of the most popular platforms like Windows, Linux, and MacOS. Most platforms can be described as a combination of the operating system and underlying hardware. The Python platform differs from compiled language platforms in that it is an interpreted, dynamic platform that runs on top of other hardware-based platforms.

The Python platform has two components:

- The Python Virtual Machine (Python VM)
- The Python Standard Library and Extended Package Ecosystem.

The Python VM is the base for the Python platform and is available across various hardware-based platforms. The Python Standard Library is a large collection of ready-made software components that provide many useful capabilities such as networking, file handling, data processing, and cryptographic operations. The Python package ecosystem, managed through pip and PyPI, extends the platform with specialized libraries for machine learning, Facial Emotion Recognition and Text Emotion Recognition. The following figure depicts a program running on the Python platform. The Python interpreter and standard library insulate the program from the underlying hardware.

. What Can Python Technology Do?

The most common types of programs written in the Python programming language include web applications, data processing systems, machine learning models, and real-time interactive systems. In the context of the proposed Multimodal Emotion Recognition (MER) system, Python serves as the core development language integrating computer vision, natural language processing, deep learning models, and response generation into a unified framework.

An application is a standalone program that runs directly on the Python platform. A special type of application known as a server supports multiple clients in real-time interaction scenarios. Examples include emotion recognition engines, chatbot backends, and inference servers handling facial and textual inputs. Another specialized program type is a modular service. A Python-based module can be considered a component that performs specific tasks such as facial emotion detection, text analysis, or response generation. These modular components improve scalability and maintainability by separating functionalities instead of building a monolithic system.

How does the Python ecosystem support all these kinds of programs? It provides a wide range of libraries and frameworks that enable efficient development and deployment. Every full implementation of the Python platform offers the following features:

• **The Essentials:** Objects, strings, threading, data structures, input/output handling, and system utilities required for building core application logic.

• **Computer Vision:** OpenCV for real-time image capture, face detection, and preprocessing of facial data for the ViT-Tiny model.

• **Natural Language Processing:** Transformers and Hugging Face libraries for implementing MiniLM to analyze textual emotions and contextual meaning.

• **Machine Learning & Deep Learning:** PyTorch and TensorFlow for building, training, and deploying models such as ViT-Tiny for FER and integrating emotion classification pipelines.

• **Audio Processing (Optional):** Librosa for extracting audio features if speech-based emotion recognition is incorporated.

• **Data Processing:** NumPy and Pandas for handling datasets, preprocessing inputs, and managing intermediate results during model inference.

• **Model Integration & APIs:** FastAPI or Flask for creating APIs that connect emotion recognition modules with response generation systems like DialoGPT.

• **Database Connectivity:** SQLite or MongoDB for storing user interactions, emotion logs, and system outputs for analysis and improvement.

• **Deployment & Scalability:** Docker for containerization and efficient deployment of the MER system across different environments, ensuring portability and scalability.

This ecosystem enables Python to act as a powerful backbone for developing an efficient, scalable, and real-time multimodal emotion-aware system.

8.2 TENSORFLOW

TensorFlow is an open-source machine learning framework developed by Google that provides powerful tools for building and deploying deep learning models. In the context of the proposed Multimodal Emotion Recognition (MER) system, TensorFlow is used to develop and support deep learning models for facial emotion recognition (FER) and assist in integrating multimodal pipelines for accurate and efficient emotion detection.

• A flexible design that enables both model training and real-time inference, making it suitable for deploying emotion recognition systems in interactive applications.
• Support for GPU acceleration, allowing faster training and efficient processing of large-scale image and text datasets.

TensorFlow enables efficient model handling by supporting optimized computation graphs and scalable deployment. In the MER system, it plays a role in processing facial data, extracting meaningful features, and integrating outputs with other modalities such as text. The framework ensures stability and performance when dealing with real-time emotion recognition tasks in dynamic environments.

1. **Model Training** – Deep learning models such as ViT-Tiny for facial emotion recognition are trained using labeled datasets containing various emotional expressions. The training process involves preprocessing image data, feeding it into the model, calculating loss, and updating weights through backpropagation. Tasks include dataset preparation, model tuning, and validation to achieve high accuracy in emotion classification.

2. **Real-Time Inference** – The trained models are deployed for real-time emotion detection, where facial inputs from a camera are processed instantly to identify emotions. TensorFlow ensures low-latency inference, enabling the system to respond quickly in applications like virtual assistants and interactive systems.

3. **Performance Visualization** – TensorBoard, TensorFlow's visualization tool, is used to monitor training metrics such as accuracy, loss, and model performance over time. This helps in analyzing model behavior, optimizing hyperparameters, and ensuring reliable performance of the emotion recognition system.

8.3 HUGGINGFACE TRANSFORMERS:

Hugging Face Transformers is an open-source library that provides state-of-the-art pre-trained models for natural language processing and multimodal AI tasks. In the context of the proposed Multimodal Emotion Recognition (MER) system, Transformers serves as the backbone for Textual Emotion Recognition (TER) and response generation by leveraging models such as MiniLM for emotion classification and DialoGPT for generating context-aware, empathetic replies.

The Transformers architecture is designed around the following key principles:
• **Pre-trained Models:** Availability of highly optimized transformer models like MiniLM and DialoGPT that can be fine-tuned for specific tasks such as emotion detection.
• **Scalability and Flexibility:** Supports easy integration of models into real-time systems, enabling both training and inference across different environments.
• **Contextual Understanding:** Utilizes attention mechanisms to capture semantic meaning, context, and emotional tone in textual inputs, improving accuracy over traditional NLP approaches.

The Transformers-based pipeline in the proposed system processes user text inputs through the MiniLM model to extract emotional context. This output is then combined with facial emotion predictions through a fusion mechanism. The fused emotional state is used to guide DialoGPT in generating responses that are empathetic and aligned with the user's current emotional condition.

1. **Textual Emotion Recognition (MiniLM)** – MiniLM is used for efficient and accurate emotion classification from textual input. The model processes tokenized text using attention mechanisms to understand context, sentiment, and intent. Its lightweight architecture ensures fast inference, making it suitable for real-time applications while maintaining strong performance in emotion detection tasks.
2. **Response Generation (DialoGPT)** – DialoGPT is employed to generate human-like conversational responses. Based on the fused emotional input, it produces context-aware replies that adapt to the user's emotional state. This enables the system to deliver empathetic interactions rather than generic responses, improving user engagement.
3. **Model Integration and Deployment** – Hugging Face Transformers provides easy-to-

use APIs for loading, fine-tuning, and deploying models. These models can be integrated into backend services using frameworks like PyTorch and served through APIs for real-time interaction. The modular design allows seamless connection between emotion recognition components and conversational agents, ensuring scalability and maintainability of the MER system.

8.4 OPENCV(COMPUTER VISION)

OpenCV is an open-source computer vision library that provides a wide range of tools for real-time image and video processing. In the context of the proposed Multimodal Emotion Recognition (MER) system, OpenCV serves as the core component for facial data acquisition and preprocessing, enabling accurate and efficient Facial Emotion Recognition (FER) using models like ViT-Tiny.

OpenCV provides the following capabilities essential to the MER framework:
• **Real-time Face Detection:** Captures live video streams from the camera and detects human faces using pre-trained classifiers, ensuring that only relevant facial regions are processed for emotion recognition.

• **Image Preprocessing:** Performs operations such as resizing, normalization, grayscale conversion, and noise reduction to prepare facial images in a format suitable for deep learning models, improving accuracy and consistency.

• **Frame Extraction and Optimization:** Efficiently extracts frames from video streams and processes them with minimal latency, enabling smooth real-time emotion detection without performance bottlenecks.

OpenCV integrates seamlessly with deep learning frameworks like TensorFlow and PyTorch, allowing preprocessed facial data to be directly fed into the ViT-Tiny model for emotion classification. It plays a critical role in maintaining system responsiveness and ensuring reliable visual input handling in dynamic, real-world environments such as virtual assistants and interactive applications.

8.5 DJANGO FRAMEWORK

Django is a high-level Python web framework that enables rapid development of secure and scalable web applications. In the context of the proposed Multimodal Emotion Recognition (MER) system, Django serves as the backend framework that manages user interactions, handles API requests, coordinates between emotion recognition modules, and delivers responses generated by the system. It ensures a structured and maintainable architecture for integrating computer vision, NLP models, and conversational components into a unified platform.

Django provides the following capabilities essential to the MER framework:

• **Robust Backend Management:** Handles user requests, manages session data, and coordinates communication between FER (OpenCV + ViT-Tiny), TER (MiniLM), and response generation (DialoGPT) modules.

• **Built-in Security Features:** Provides protection against common vulnerabilities such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF), ensuring safe handling of sensitive user data including facial inputs and text interactions.

• **Scalable Web Architecture:** Supports modular application design, allowing different components of the MER system to be developed, deployed, and maintained efficiently.

1. **Modular System Integration** –The MER system is structured into multiple Django apps, each responsible for specific functionalities such as user input handling, emotion processing, and response delivery. Django acts as the central controller that routes requests to the appropriate modules and aggregates outputs from FER and TER pipelines before sending them to the user interface.
2. **API Development and Communication** –Django (with Django REST Framework) is used to build RESTful APIs that facilitate communication between frontend interfaces and backend services. These APIs handle tasks such as receiving facial frames and text inputs, triggering emotion recognition models, and returning fused emotional states along with generated responses in real time.
3. **Data Management and Logging** –Django's ORM (Object-Relational Mapping) is used to manage databases for storing user interactions, detected emotions, and system responses. This enables efficient querying, logging, and analysis of data for improving model performance and user experience over time.

8.6 SCIKIT-LEARN

Scikit-learn is an open-source machine learning library in Python that provides efficient tools for data analysis, preprocessing, and classical machine learning algorithms. In the context of the proposed Multimodal Emotion Recognition (MER) system, Scikit-learn is utilized to support data preprocessing, feature engineering, model evaluation, and auxiliary learning tasks that enhance the performance and reliability of emotion recognition models.

Scikit-learn enables efficient handling of structured data and complements deep learning models by providing lightweight algorithms and utilities for improving system performance. It plays a supporting role in preparing input data, validating model outputs, and performing analytical tasks on emotion prediction results.

1. **Data Preprocessing and Feature Engineering** – Scikit-learn is used to preprocess input data by performing operations such as normalization, scaling, encoding, and feature extraction. For textual inputs, it can assist in vectorization techniques like TF-IDF, while for system-level analysis, it helps structure intermediate outputs from FER and TER models. This ensures that data is in an optimal format for accurate emotion classification.
2. **Model Evaluation and Validation** – Scikit-learn provides evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix to assess the performance of emotion recognition models. These metrics help in comparing unimodal and multimodal approaches, validating improvements achieved through fusion mechanisms, and ensuring the reliability of the system.
3. **Anomaly Detection and Analysis** – Scikit-learn can be used to implement lightweight anomaly detection techniques to identify unusual patterns in user interactions or emotion predictions. This helps in detecting inconsistent inputs, model misclassifications, or abnormal behavior in the system, contributing to improved robustness and reliability of the MER framework.

9.1 SOURCE CODE

```
import os
import cv2
import numpy as np
import librosa
from tensorflow import keras
from tensorflow.keras import layers
from django.conf import settings
from django.shortcuts import render
from django.contrib.auth.decorators import login_required
from django.db.models import Avg
from facial_emotion.models import FacialEmotionCapture
from voice_emotion.models import VoiceEmotionAnalysis
from recommendations.models import RecommendationItem



class FacialEmotionDetector:

    def __init__(self):
        """
        Initialize model and face detection system
        """
        self.model_path = os.path.join(
            settings.BASE_DIR,
            'facial_emotion',
            'ml_models',
            'facial_emotion_model.h5'
        )

        if not os.path.exists(self.model_path):
            raise FileNotFoundError(f"Model not found at {self.model_path}")

        self.model = keras.models.load_model(self.model_path)
        self.emotion_labels = settings.EMOTION_CLASSES

        # Load Haar Cascade for face detection
        self.face_cascade = cv2.CascadeClassifier(
            cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
```

```
    )

    if self.face_cascade.empty():
        raise RuntimeError("Face cascade not loaded properly")

# ----------------------------------------------------
def preprocess_face(self, face_img):
    """
    Preprocess image for model input
    """
    face_img = cv2.resize(face_img, (48, 48))
    face_img = cv2.cvtColor(face_img, cv2.COLOR_BGR2GRAY)

    face_img = face_img.astype("float32") / 255.0
    face_img = face_img.reshape(1, 48, 48, 1)

    return face_img

# ----------------------------------------------------
def detect_faces(self, frame):
    """
    Detect faces in frame
    """
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray = cv2.equalizeHist(gray)

    faces = self.face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(48, 48)
    )

    return faces

# ----------------------------------------------------
def predict_emotion(self, face_img):
    """
    Predict emotion from face
    """
```

```
        processed = self.preprocess_face(face_img)

        predictions = self.model.predict(processed, verbose=0)[0]
        index = np.argmax(predictions)

        emotion = self.emotion_labels[index]
        confidence = float(predictions[index])

        return emotion, confidence, predictions

    # ----------------------------------------------------
    def process_frame(self, frame):
        """
        Full pipeline: detect → predict → return structured output
        """
        results = []
        faces = self.detect_faces(frame)

        for (x, y, w, h) in faces:
            roi = frame[y:y+h, x:x+w]

            if roi.size == 0:
                continue

            emotion, confidence, all_preds = self.predict_emotion(roi)

            results.append({
                "emotion": emotion,
                "confidence": confidence,
                "coordinates": {
                    "x": int(x),
                    "y": int(y),
                    "width": int(w),
                    "height": int(h)
                }
            })

        return results
```

```
def extract_audio_features(file_path, duration=3):
    """
    Extract multiple audio features using librosa
    """
    try:
        audio, sr = librosa.load(file_path, duration=duration, sr=22050)

        if len(audio) < 1024:
            return None

        # MFCC
        mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=40)
        mfcc_mean = np.mean(mfcc.T, axis=0)

        # Chroma
        chroma = librosa.feature.chroma_stft(y=audio, sr=sr)
        chroma_mean = np.mean(chroma.T, axis=0)

        # Mel Spectrogram
        mel = librosa.feature.melspectrogram(y=audio, sr=sr)
        mel_mean = np.mean(mel.T, axis=0)

        # Zero Crossing Rate
        zcr = librosa.feature.zero_crossing_rate(audio)
        zcr_mean = np.mean(zcr)

        # Combine features
        features = np.hstack([
            mfcc_mean,
            chroma_mean,
            mel_mean,
            [zcr_mean]
        ])

        return features

    except Exception as e:
        print(f"Audio processing error: {e}")
        return None
```

```
def create_voice_model(input_shape, num_classes):
    """
    CNN + LSTM Hybrid Model
    """
    model = keras.Sequential()

    # Convolution Layer
    model.add(layers.Conv1D(128, 5, activation='relu', input_shape=input_shape))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling1D(2))
    model.add(layers.Dropout(0.3))

    # LSTM Layer
    model.add(layers.LSTM(64))
    model.add(layers.Dropout(0.3))

    # Dense Layers
    model.add(layers.Dense(128, activation='relu'))
    model.add(layers.Dropout(0.4))
    model.add(layers.Dense(num_classes, activation='softmax'))

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.0001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model



def seed_recommendations():
    """
    Insert predefined recommendations into database
    """
    data = [
        {
```

```
            "title": "Meditation Guide",
            "description": "Calm your mind and relax",
            "category": "activity",
            "target_emotions": ["sad", "angry"]
        },
        {
            "title": "Comedy Movies",
            "description": "Light-hearted entertainment",
            "category": "video",
            "target_emotions": ["happy"]
        },
        {
            "title": "Motivational Articles",
            "description": "Boost your mindset",
            "category": "article",
            "target_emotions": ["sad", "neutral"]
        }
    ]

    RecommendationItem.objects.all().delete()

    for item in data:
        RecommendationItem.objects.create(**item)



@login_required
def admin_dashboard(request):
    """
    Display system analytics
    """

    total_facial = FacialEmotionCapture.objects.count()
    total_voice = VoiceEmotionAnalysis.objects.count()

    avg_facial_conf = FacialEmotionCapture.objects.aggregate(
        Avg('confidence_score')
    )['confidence_score__avg'] or 0

    avg_voice_conf = VoiceEmotionAnalysis.objects.aggregate(
```

```
        Avg('confidence_score')
    )['confidence_score__avg'] or 0

    total = total_facial + total_voice

    context = {
        "total_facial": total_facial,
        "total_voice": total_voice,
        "total": total,
        "avg_facial_conf": round(avg_facial_conf, 2),
        "avg_voice_conf": round(avg_voice_conf, 2)
    }

    return render(request, "admin/dashboard.html", context)



if __name__ == "__main__":
    print("=" * 50)
    print("MULTIMODAL EMOTION RECOGNITION SYSTEM")
    print("=" * 50)

    print("Modules Loaded Successfully")
```

10.RESULTS/DISCUSSIONS

10.1 SYSTEM TESTING

System testing is conducted to ensure that the **AI-Driven Personalized Learning System** functions correctly, efficiently, and securely according to the specified requirements. Testing verifies whether each module operates as intended and whether the integrated system meets performance, usability, and security standards.

Testing is performed at multiple levels including **Unit Testing, Integration Testing, System Testing, and User Acceptance Testing (UAT).**

TESTING OBJECTIVES

The main objectives of system testing are:

- To verify that all functional requirements are implemented correctly.
- To ensure accurate AI-based content generation and feedback mechanisms.
- To validate secure authentication and role-based access control.
- To confirm that dashboards and analytics display correct information.
- To test system performance under multiple user conditions.
- To ensure data privacy, integrity, and reliability.

10.2 TYPES OF TESTING

UNIT TESTING:

Unit testing is performed on individual modules independently to verify correct functionality and expected outputs.

Modules tested include:

- User Registration and Login Module
- Content Management Module
- AI Integration Module
- Assessment Module
- Dashboard Module

Example:

The login module is tested with valid and invalid credentials to ensure secure authentication.

INTEGRATION TESTING:

Integration testing verifies that different modules work together properly without data loss or processing errors.

Examples include:

- Student login followed by dashboard loading
- Content upload followed by AI content generation
- Assessment submission followed by instant feedback display

This ensures smooth communication between system components.

SYSTEM TESTING

System testing evaluates the complete integrated system to verify overall functionality.

It checks:

- End-to-end workflow from login to progress tracking
- Real-time AI task generation
- Data storage and retrieval accuracy
- Report generation and export functionality

The system is tested in a simulated classroom environment to validate real-world usage.

USER ACCEPTANCE TESTING (UAT)

User Acceptance Testing is conducted with sample users (students and teachers) to ensure the system meets educational needs.

The following aspects are evaluated:

- Ease of navigation
- Accuracy of AI-generated content
- Clarity of feedback
- Dashboard usability

- Overall user satisfaction

Feedback collected during UAT is used to refine the system.

PERFORMANCE TESTING

Performance testing ensures the system responds efficiently under load.

The system is tested for:

- Response time for AI-generated tasks
- Dashboard loading speed
- Simultaneous user handling capacity
- Server performance under multiple classroom access

The system generates personalized learning content within seconds to support real-time classroom interaction.

SECURITY TESTING

Security testing ensures that sensitive academic and personal data is protected.

Security checks include:

- Password encryption validation
- Role-based access verification
- Unauthorized login attempt detection
- Data transmission encryption
- Audit log verification

The system prevents unauthorized access and protects student records.

USABILITY TESTING

Usability testing evaluates user interface simplicity and navigation efficiency.

It ensures:

- Clear menu structure
- Intuitive dashboard layout
- Easy content upload process

- Simple goal-setting interface for students

The interface is designed to be accessible for K–12 students and educators with minimal technical expertise.

a. TEST CASES

Test cases:

Test Case ID	Test Case Name	Module	Input	Expected Output	Actual Output	Status
TC_01	Valid User Login	Authentication Module	Correct username & password	User successfully logged in and dashboard displayed	As Expected	Pass
TC_02	Invalid User Login	Authentication Module	Incorrect password	Error message displayed: "Invalid Credentials"	As Expected	Pass
TC_03	Role-Based Access Control	User Management	Student tries to access Admin panel	Access denied message displayed	As Expected	Pass
TC_04	Content Upload by Teacher	Content Management	Upload PDF assignment file	File successfully uploaded and stored	As Expected	Pass
TC_05	AI Personalized Quiz Generation	AI Module	Student performance data	Personalized quiz generated based on difficulty level	As Expected	Pass
TC_06	Instant Feedback Generation	Assessment Module	Student submits quiz answers	Immediate score and feedback displayed	As Expected	Pass
TC_07	Dashboard Performance Display	Dashboard Module	Student logs into dashboard	Progress graphs and performance metrics displayed correctly	As Expected	Pass

TC_08	Engagement Tracking	Engagement Module	Student completes learning task	Time spent and completion status recorded in database	As Expected	Pass
TC_09	Report Generation & Export	Reporting Module	Teacher selects class report	Report generated and downloadable in PDF/Excel format	As Expected	Pass
TC_10	Unauthorized Data Access Attempt	Security Module	Direct URL access without login	System redirects to login page	As Expected	Pass

11.SCREENSHOTS

FIG1: LOGIN PAGE

The This screen allows users to securely log in to the system using their credentials. It ensures authentication before accessing emotion analysis features and dashboards.

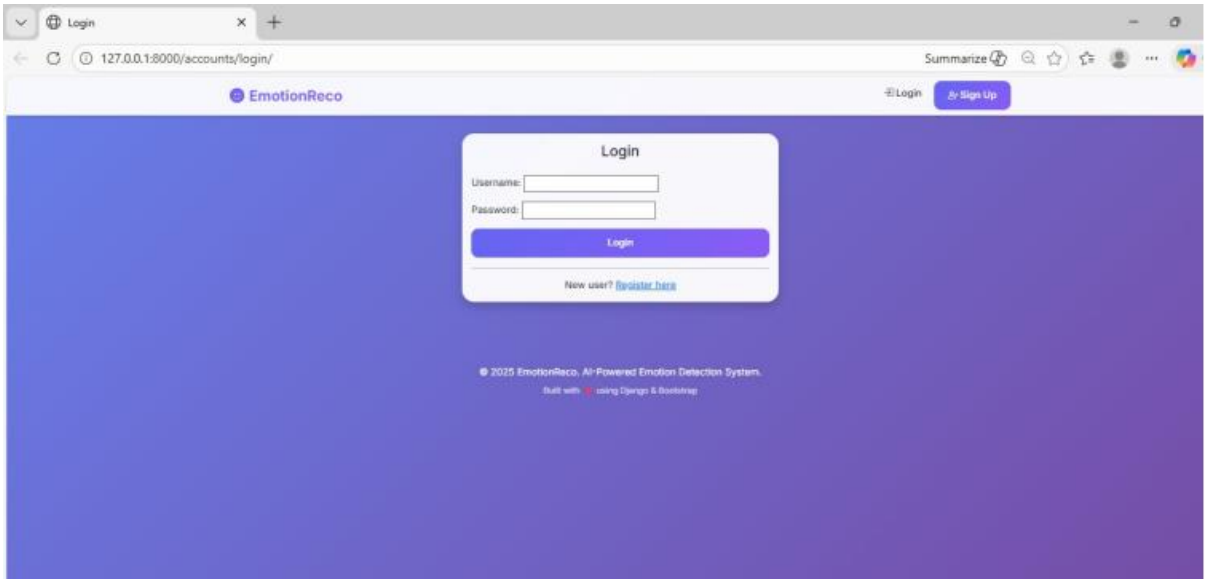


FIG2: REGISTER PAGE

This page enables new users to create an account by providing required details.

It ensures secure user onboarding and stores credentials for future authentication.

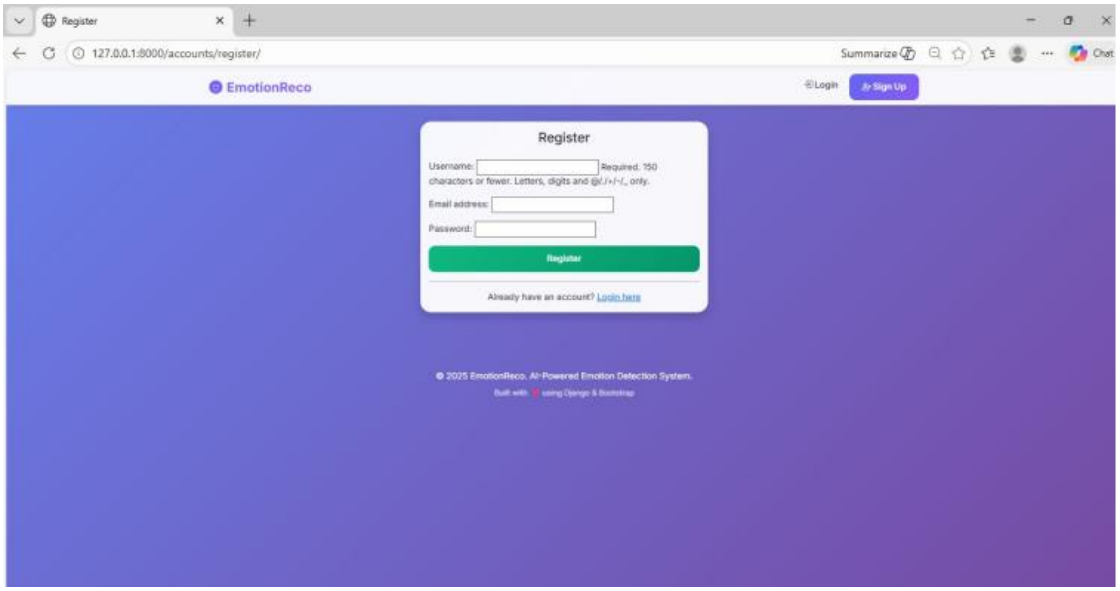


FIG3: ADMIN PANEL

This interface provides administrators with control over system operations and monitoring.

It includes access to user management, analytics, and system performance overview.

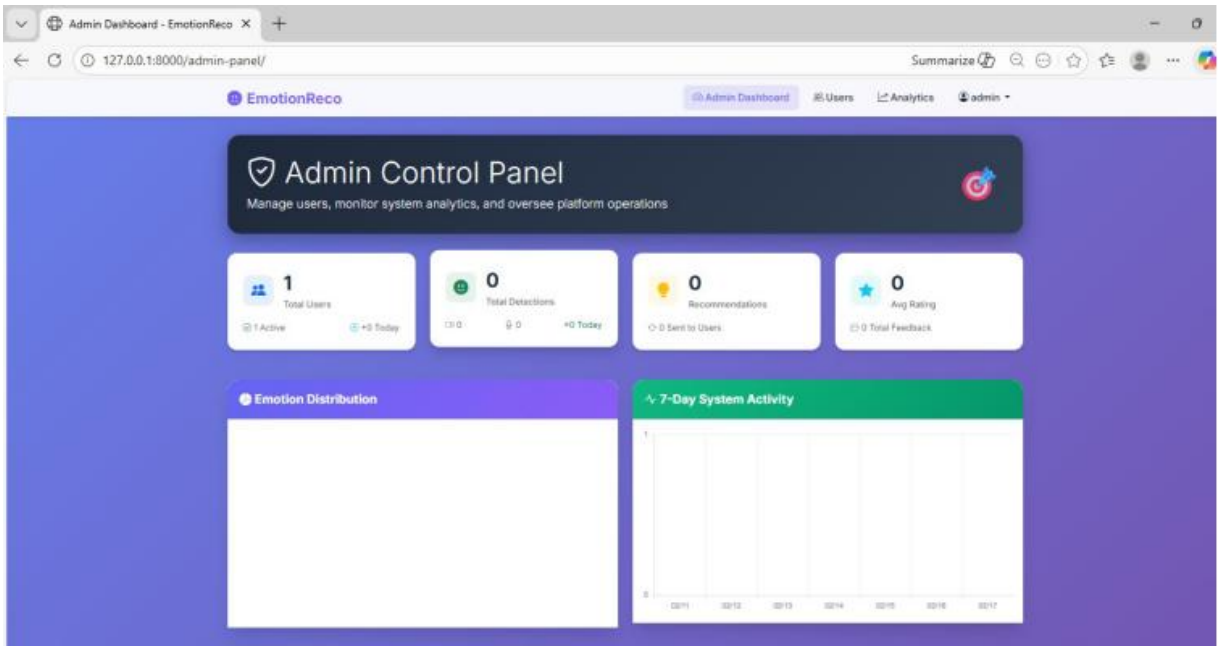


FIG4: USER MANAGEMENT

This screen allows the admin to view, manage, and control user accounts.

It supports operations like adding, updating, or removing users and managing permissions

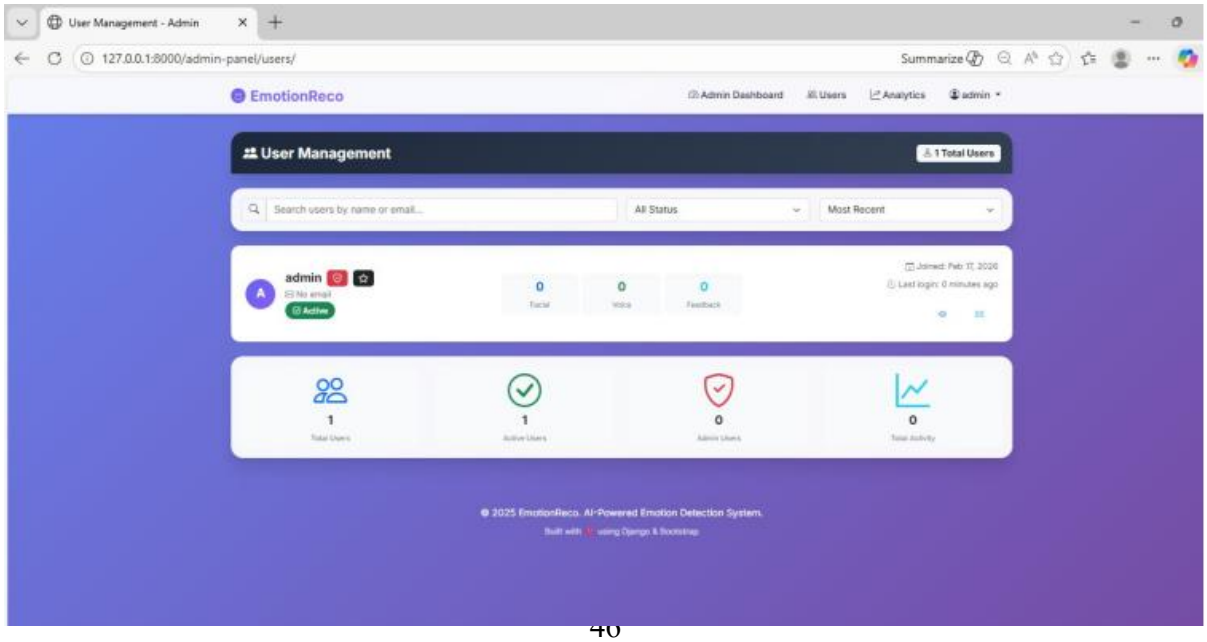


FIG5: SYSTEM ANALYTICS

This dashboard displays statistical insights such as emotion distribution and usage trends.

It helps administrators analyze system performance and user interaction patterns.

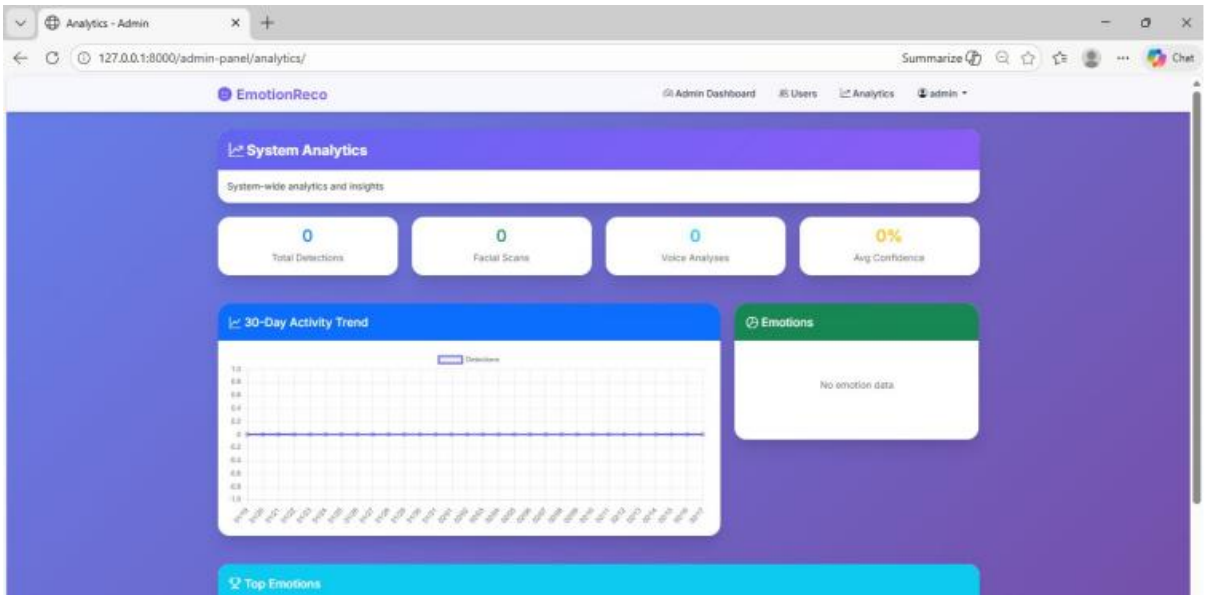


FIG6: FACIAL EMOTION DETECTION

This screen shows real-time emotion detection from facial inputs using the FER model.

It highlights detected emotions along with confidence scores from image analysis.

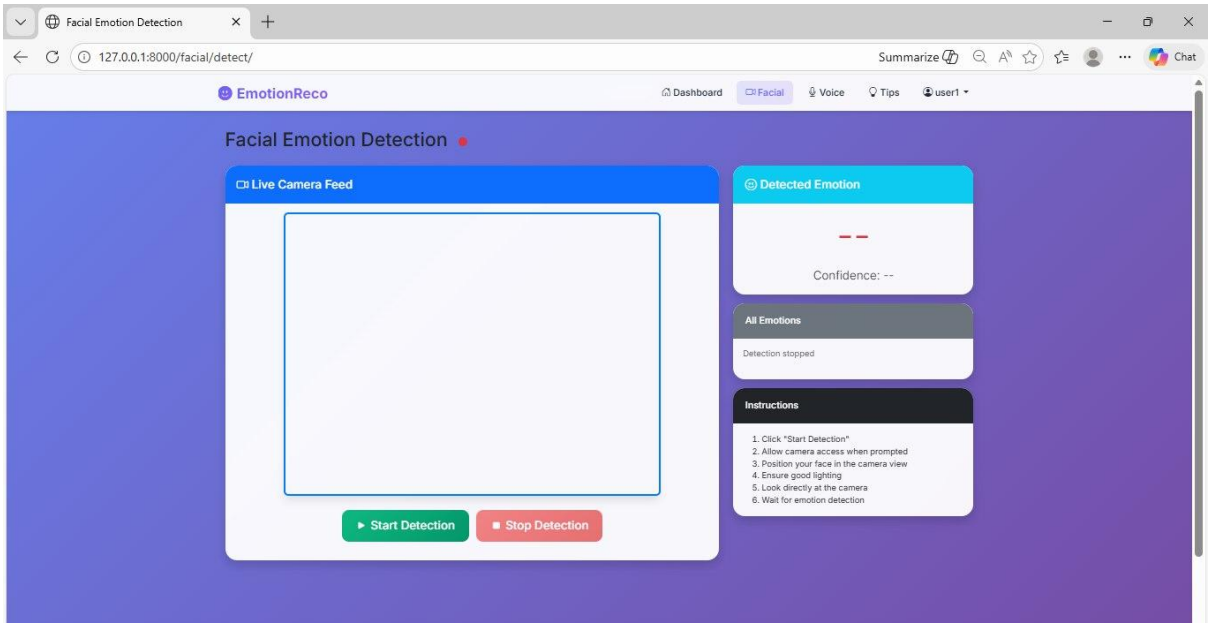


FIG7: VOICE EMOTION ANALYSIS

This interface processes audio input to detect emotions using extracted speech features. It displays the predicted emotional state based on voice characteristics.

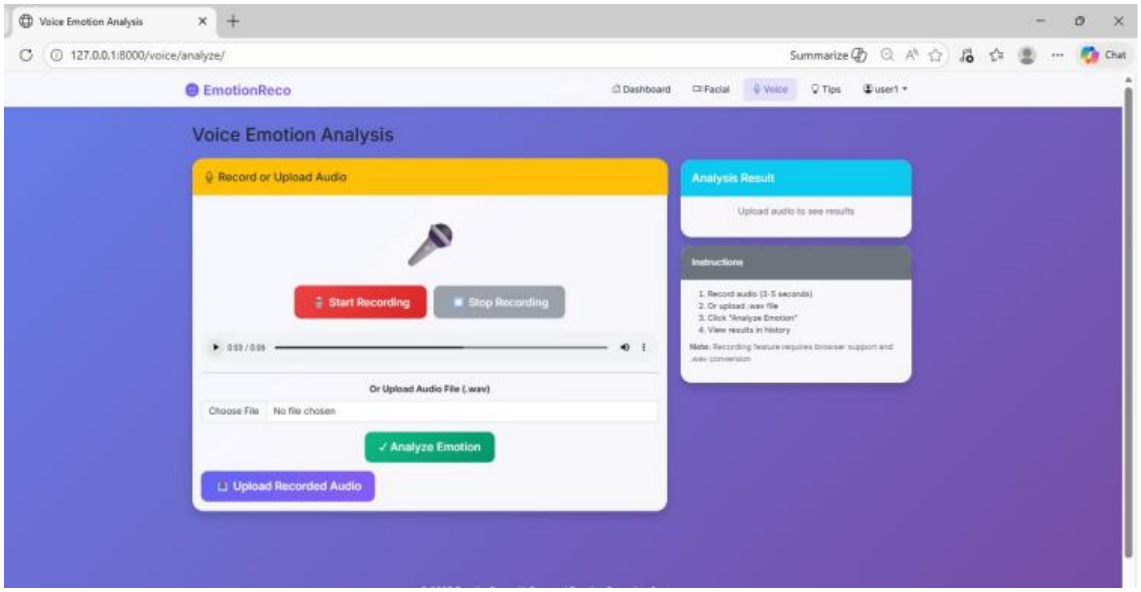
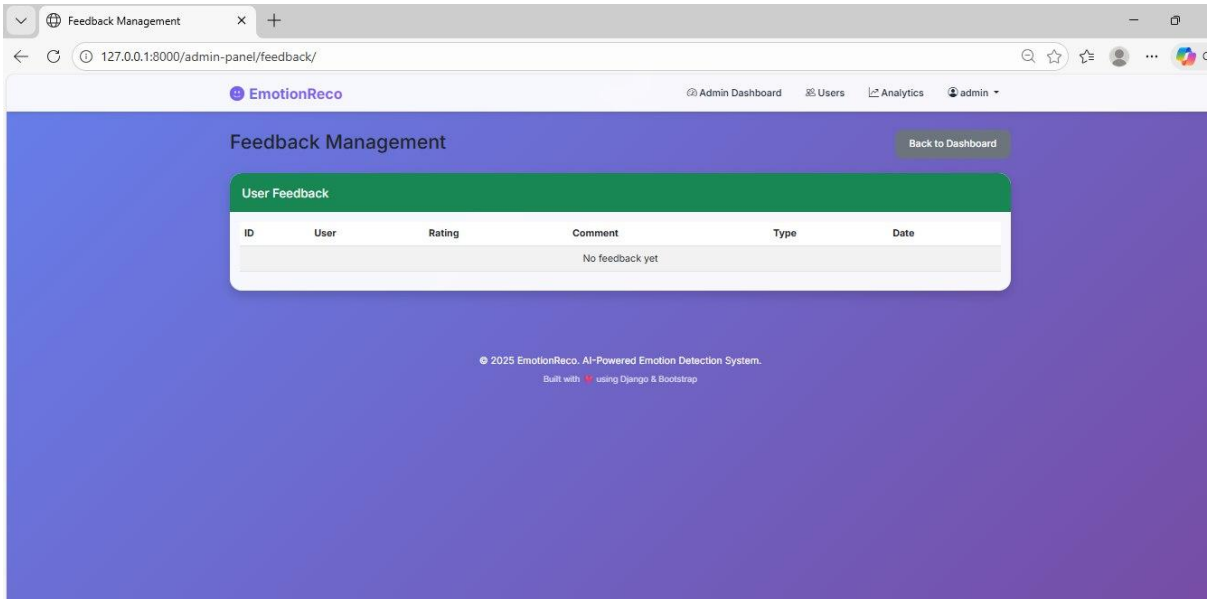


FIG8: FEEDBACK MANAGEMENT

This screen allows admins to view and manage user feedback on system responses. It helps improve system performance by analyzing user ratings and suggestions.



12. CONCLUSION

The enhancement of virtual assistants through multimodal AI for emotion recognition represents a significant advancement in human-computer interaction. By integrating both facial emotion recognition (FER) and textual emotion recognition (TER), the proposed system achieves a more holistic and accurate understanding of user emotions than traditional single-modal approaches. The experimental results, with FER achieving 71% real-time accuracy and TER achieving 59% validation accuracy, demonstrate the effectiveness of combining multiple modalities for robust emotion detection.

The system architecture emphasizes lightweight infrastructure, enabling real-time inference without compromising accuracy. This ensures that interactions remain fluid and responsive, which is critical for applications requiring timely responses, such as healthcare, education, and customer support. Additionally, by coupling emotion recognition with a DialoGPT-based response generator, the system does not merely detect emotions but also produces contextually appropriate and empathetic replies. This combination of emotion detection and intelligent response generation addresses a significant limitation of existing systems, which often fail to provide meaningful or human-like interaction.

Moreover, the proposed methodology exhibits flexibility and scalability, allowing it to adapt across various domains, from entertainment and gaming to professional and therapeutic applications. By leveraging multimodal inputs, the system is better equipped to recognize subtle or complex emotional cues, improving overall user satisfaction and engagement. In conclusion, this study validates that multimodal AI can significantly enhance the emotional intelligence of virtual assistants, paving the way for more natural, empathetic, and human-like interactions in digital environments.

b. FUTURE SCOPE

While the current system demonstrates substantial improvements over existing single-modal emotion recognition systems, there are several avenues for future enhancement to increase accuracy, responsiveness, and versatility. One primary enhancement is the integration of **additional modalities**, such as speech-based emotion recognition and physiological signals (e.g., heart rate, galvanic skin response). Incorporating audio cues can capture tone, pitch, and other paralinguistic features, which often carry strong emotional signals, particularly for subtle or mixed emotions that may not be apparent in facial expressions or text alone. Physiological signals can further augment the system's accuracy in detecting internal emotional states that are not externally expressed.

Another enhancement is the application of **advanced deep learning and transformer-based multimodal fusion techniques**. Current fusion strategies can be improved with attention-based models that dynamically weight the contribution of each modality according to the context, thereby improving performance in ambiguous or noisy situations. Additionally, leveraging **pretrained large multimodal models** can significantly reduce training time while improving the system's ability to generalize across diverse inputs and domains.

Scalability and deployment can also be enhanced by adopting **edge computing and cloud-assisted architectures**. Real-time emotion recognition on low-resource devices may be challenging, and distributing computational load between the device and the cloud can maintain responsiveness while enabling more sophisticated models. Moreover, **personalization features** could be incorporated, allowing the virtual assistant to learn user-specific emotional patterns over time, thereby tailoring responses for greater empathy and user satisfaction.

Finally, the system can be extended with **continuous learning mechanisms** and **feedback loops**, where user feedback directly informs model adjustments in near real-time. This would allow the virtual assistant to adapt to changing emotional contexts and evolving communication patterns, enhancing reliability and emotional intelligence. Future iterations could also explore multilingual and cross-cultural emotion recognition capabilities, ensuring broader applicability and inclusivity.

In summary, while the current multimodal system lays a strong foundation for

emotionally aware virtual assistants, incorporating additional modalities, advanced fusion techniques, personalization, and continuous learning will further refine its ability to interact naturally, empathetically, and effectively with users across diverse scenarios.

13.REFERENCES

[1] P. Ekman, *Emotions Revealed: Recognizing Faces and Feelings to Improve Communication and Emotional Life*, New York: Times Books, 2003.

[2] M. Pantic and L. J. M. Rothkrantz, "Automatic analysis of facial expressions: The state of the art," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 12, pp. 1424–1445, Dec. 2000.

[3] R. Cowie, E. Douglas-Cowie, N. Tsapatsoulis, G. Votsis, S. Kollias, W. Fellenz, and J. G. Taylor, "Emotion recognition in human-computer interaction," *IEEE Signal Processing Magazine*, vol. 18, no. 1, pp. 32–80, Jan. 2001.

[4] T. Baltrušaitis, C. Ahuja, and L. Morency, "Multimodal machine learning: A survey and taxonomy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 2, pp. 423–443, Feb. 2019.

[5] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, Oct. 2016.

[6] D. Kollias, S. Zafeiriou, "Deep affect: Facial expression recognition using deep learning," *Computer Vision and Image Understanding*, vol. 171, pp. 12–22, 2018.

[7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[8] R. Calvo and S. D'Mello, "Affect detection: An interdisciplinary review of models, methods, and their applications," *IEEE Transactions on Affective Computing*, vol. 1, no. 1, pp. 18–37, Jan.–June 2010.

[9] T. Young, D. Hazarika, S. Poria, and E. Cambria, "Recent trends in deep learning based natural language processing," *IEEE Computational Intelligence Magazine*, vol. 13, no. 3, pp. 55–75, Aug. 2018.

[10] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, 2019. [Online]. Available: https://openai.com/research